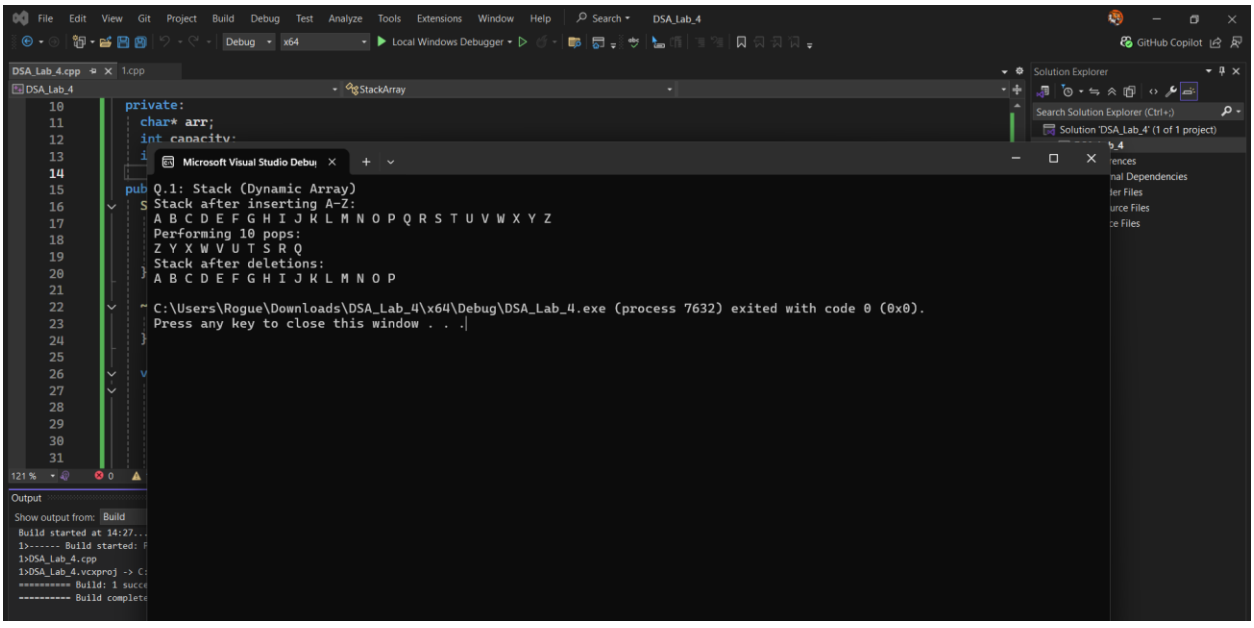
Name : Prashant Ranjan Roll No : 24MC3035

```
/*
Q.1: Implement the stack considering the dynamic array and perform the
following operations:
(a) Insert the 26 Alphabets, i.e., "A"...."Z", into the stack
(b) Perform ten consecutive deletions into the stack */

#include <iostream>
using namespace std;

class StackArray {
private:
  char* arr;
  int capacity;
  int topIndex;

public:
  StackArray(int cap = 10) {
    capacity = cap;
    arr = new char[capacity];
    topIndex = -1;
  }

  ~StackArray() {
    delete[] arr;
  }

  void push(char element) {
    if (topIndex == capacity - 1) {
      char* newArr = new char[capacity * 2];
      for (int i = 0; i <= topIndex; i++) newArr[i] = arr[i];
      delete[] arr;
      arr = newArr;
      capacity *= 2;
    }
    arr[++topIndex] = element;
  }

  char pop() {
    if (isEmpty()) throw runtime_error("Stack is empty!");
    return arr[topIndex--];
  }

  bool isEmpty() const {
    return topIndex == -1;
  }

  void display() const {
    for (int i = 0; i <= topIndex; i++) cout << arr[i] << " ";
    cout << endl;
  }
};
```

```
int main() {
  cout << "Q.1: Stack (Dynamic Array)" << endl;
  StackArray s;

  for (char c = 'A'; c <= 'Z'; c++) s.push(c);

  cout << "Stack after inserting A-Z:" << endl;
  s.display();

  cout << "Performing 10 pops:" << endl;
  for (int i = 0; i < 10; i++) cout << s.pop() << " ";
  cout << endl;

  cout << "Stack after deletions:" << endl;
  s.display();
}
```

Output:

```
Q.1: Stack (Dynamic Array)
Stack after inserting A-Z:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Performing 10 pops:
Z Y X W V U T S R Q
Stack after deletions:
A B C D E F G H I J K L M N O P
```



```
/*
Q.2: Implement the stack considering the singly linked list implementation
and perform the following operations:
(a) Insert the 26 Alphabets, i.e., "A"...."Z", into the stack
(b) Perform ten consecutive deletions into the stack
*/
```

```
#include <iostream>
using namespace std;

struct Node {
  char data;
  Node* next;
  Node(char d) : data(d), next(nullptr) {}
};

class StackLinkedList {
private:
  Node* topNode;

public:
  StackLinkedList() : topNode(nullptr) {}

  ~StackLinkedList() {
    while (!isEmpty()) pop();
  }

  void push(char element) {
    Node* newNode = new Node(element);
    newNode->next = topNode;
    topNode = newNode;
  }

  char pop() {
    if (isEmpty()) throw runtime_error("Stack is empty!");
    Node* temp = topNode;
    char val = temp->data;
    topNode = topNode->next;
    delete temp;
    return val;
  }

  bool isEmpty() const {
    return topNode == nullptr;
  }

  void display() const {
    Node* temp = topNode;
    while (temp) {
      cout << temp->data << " ";
      temp = temp->next;
    }
    cout << endl;
  }
};

int main() {
  cout << "Q.2: Stack (Linked List)" << endl;
  StackLinkedList s;

  for (char c = 'A'; c <= 'Z'; c++) s.push(c);
```

```
  cout << "Stack after inserting A-Z:" << endl;
  s.display();

  cout << "Performing 10 pops:" << endl;
  for (int i = 0; i < 10; i++) cout << s.pop() << " ";
  cout << endl;

  cout << "Stack after deletions:" << endl;
  s.display();
}
```

Output:

Q.2: Stack (Linked List)

Stack after inserting A-Z:

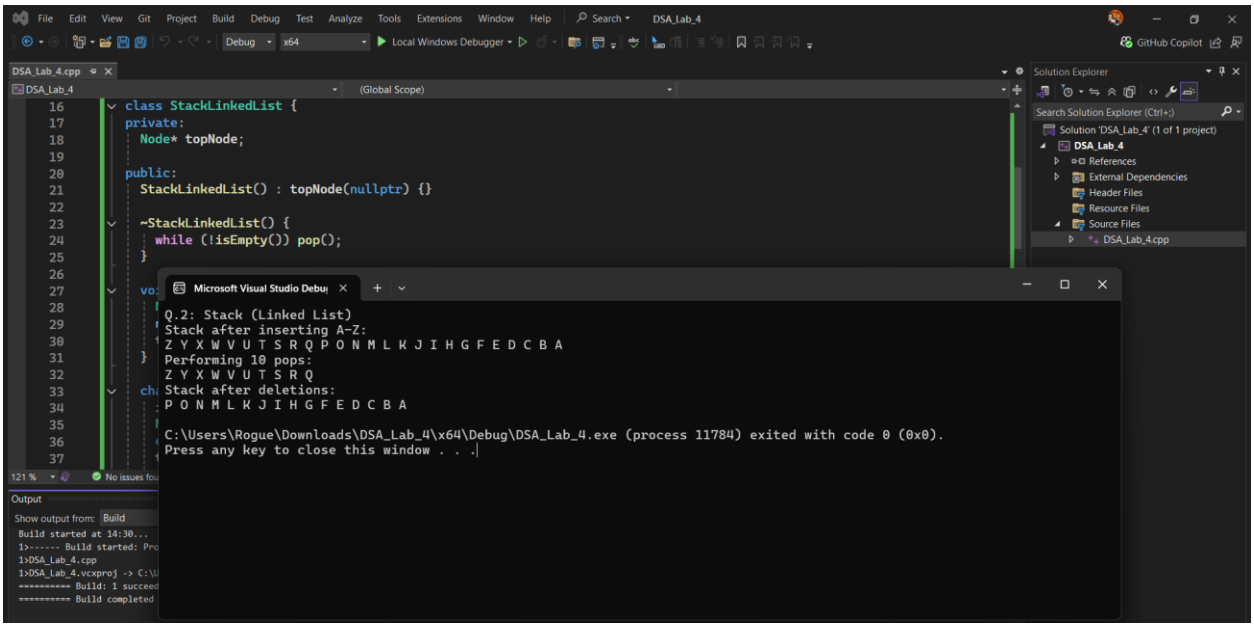
Z Y X W V U T S R Q P O N M L K J I H G F E D C B A

Performing 10 pops:

Z Y X W V U T S R Q

Stack after deletions:

P O N M L K J I H G F E D C B A



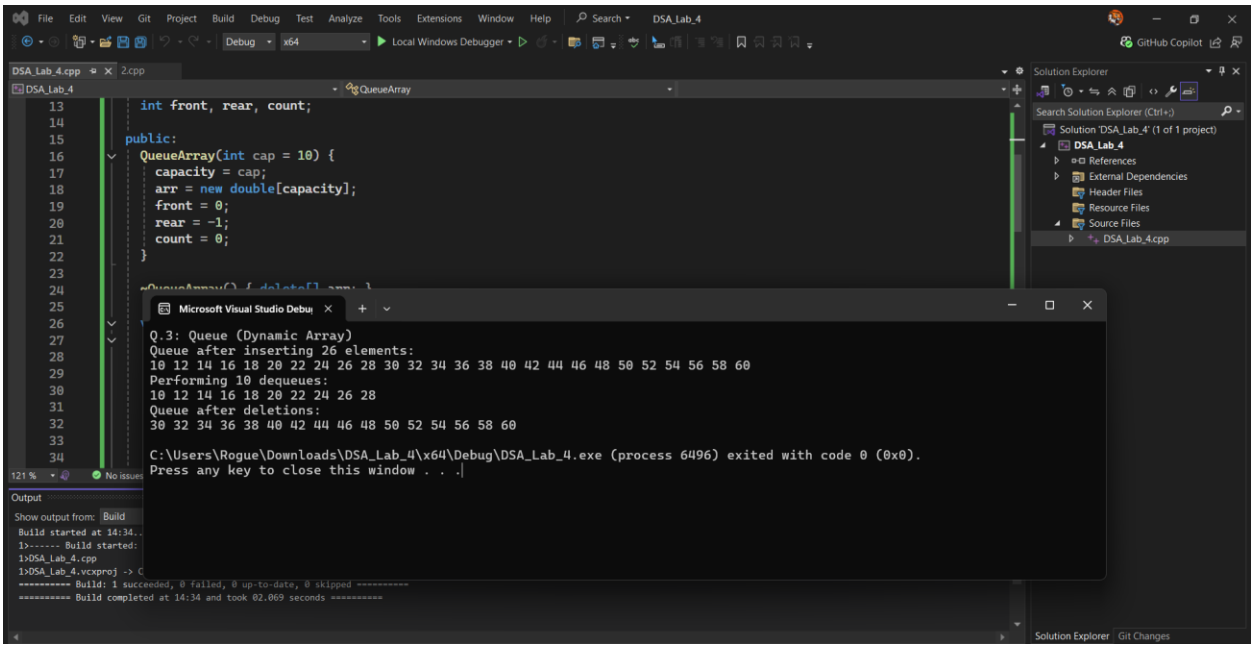
```
/*Q.3: Implement the Queue considering the dynamic array and perform the
following operations:
(a) Insert the 26 real numbers, i.e., 10.0, 12.0,..., into the Queue
(b) Perform ten consecutive deletions into the Queue
*/
```

```
#include <iostream>
using namespace std;

class QueueArray {
private:
  double* arr;
  int capacity;
  int front, rear, count;

public:
  QueueArray(int cap = 10) {
    capacity = cap;
    arr = new double[capacity];
    front = 0;
    rear = -1;
    count = 0;
  }

  ~QueueArray() { delete[] arr; }

  void enqueue(double element) {
    if (count == capacity) {
      // Resize
      double* newArr = new double[capacity * 2];
      for (int i = 0; i < count; i++) newArr[i] = arr[(front + i) %
capacity];
      delete[] arr;
      arr = newArr;
      front = 0;
      rear = count - 1;
      capacity *= 2;
    }
    rear = (rear + 1) % capacity;
    arr[rear] = element;
    count++;
  }

  double dequeue() {
    if (isEmpty()) throw runtime_error("Queue is empty!");
    double val = arr[front];
    front = (front + 1) % capacity;
    count--;
    return val;
  }

  bool isEmpty() const { return count == 0; }

  void display() const {
    for (int i = 0; i < count; i++) {
      cout << arr[(front + i) % capacity] << " ";
    }
    cout << endl;
  }
```

```
};

int main() {
  cout << "Q.3: Queue (Dynamic Array)" << endl;
  QueueArray q;

  for (int i = 0; i < 26; i++) q.enqueue(10.0 + i * 2.0);

  cout << "Queue after inserting 26 elements:" << endl;
  q.display();

  cout << "Performing 10 dequeues:" << endl;
  for (int i = 0; i < 10; i++) cout << q.dequeue() << " ";
  cout << endl;

  cout << "Queue after deletions:" << endl;
  q.display();
}
```

Output:

```
Q.3: Queue (Dynamic Array)

Queue after inserting 26 elements:

10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58
60

Performing 10 dequeues:

10 12 14 16 18 20 22 24 26 28

Queue after deletions:

30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60
```



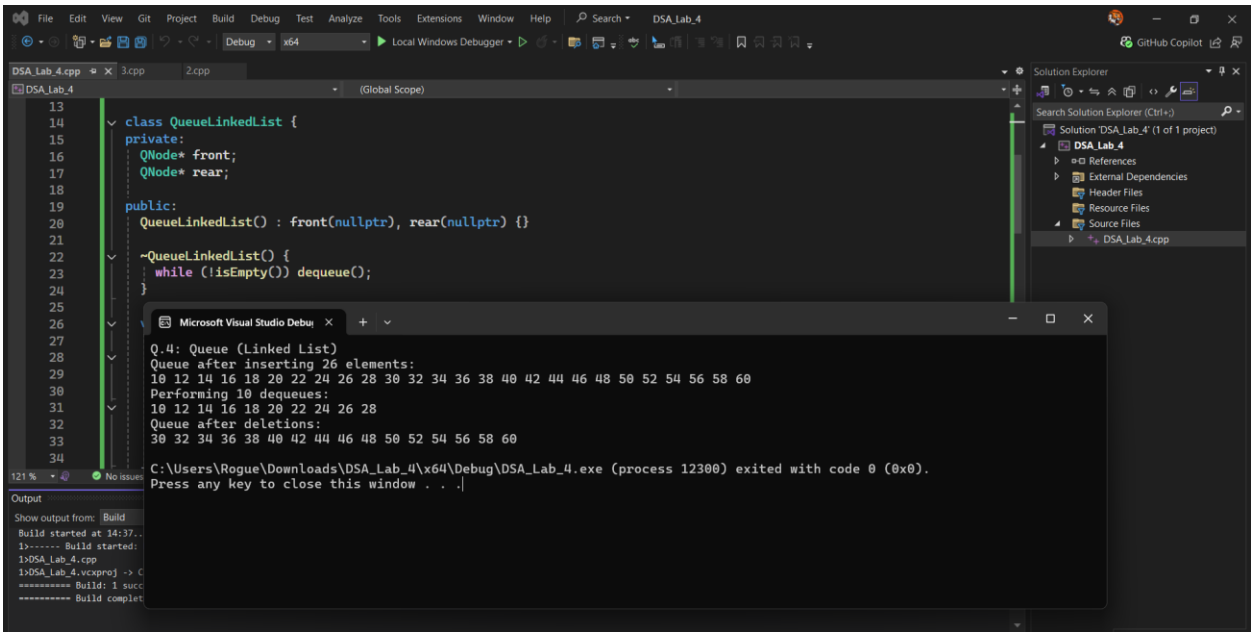

```
/*Q.4: Implement the Queue considering the singly linked list
implementation and perform the following operations:
(a) Insert the 26 real numbers, i.e., 10.0, 12.0,..., into the Queue
(b) Perform ten consecutive deletions into the Queue*/

#include <iostream>
using namespace std;

struct QNode {
  double data;
  QNode* next;
  QNode(double d) : data(d), next(nullptr) {}
};

class QueueLinkedList {
private:
  QNode* front;
  QNode* rear;

public:
  QueueLinkedList() : front(nullptr), rear(nullptr) {}

  ~QueueLinkedList() {
    while (!isEmpty()) dequeue();
  }

  void enqueue(double element) {
    QNode* newNode = new QNode(element);
    if (isEmpty()) {
      front = rear = newNode;
    }
    else {
      rear->next = newNode;
```

```
      rear = newNode;
    }
  }

  double dequeue() {
    if (isEmpty()) throw runtime_error("Queue is empty!");
    QNode* temp = front;
    double val = temp->data;
    front = front->next;
    if (!front) rear = nullptr;
    delete temp;
    return val;
  }

  bool isEmpty() const { return front == nullptr; }

  void display() const {
    QNode* temp = front;
    while (temp) {
      cout << temp->data << " ";
      temp = temp->next;
    }
    cout << endl;
  }
};

int main() {
  cout << "Q.4: Queue (Linked List)" << endl;
  QueueLinkedList q;

  for (int i = 0; i < 26; i++) q.enqueue(10.0 + i * 2.0);

  cout << "Queue after inserting 26 elements:" << endl;
  q.display();

  cout << "Performing 10 dequeues:" << endl;
  for (int i = 0; i < 10; i++) cout << q.dequeue() << " ";
  cout << endl;

  cout << "Queue after deletions:" << endl;
  q.display();
}
```

Output:

```
Q.4: Queue (Linked List)

Queue after inserting 26 elements:

10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58
60

Performing 10 dequeues:
```

```
10 12 14 16 18 20 22 24 26 28

Queue after deletions:

30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60
```




```
/*
Q.5: Write a program to check whether the input arithmetic expression has
balanced symbols/brackets or not (Application of stack)? Show your output
for the following test cases:
(a) [{(a+b)/(a-b)}+{{d*(e+f)}]
(b)
[{(a+b)/(a-b)+(d*f)}+{{d*(e+f)}}]
*/
#include <iostream>
#include <string>
using namespace std;

class Stack {
private:
  char* arr;
  int capacity, topIndex;

public:
  Stack(int cap = 10) {
    capacity = cap;
    arr = new char[capacity];
    topIndex = -1;
  }

  ~Stack() { delete[] arr; }

  void push(char c) {
    if (topIndex == capacity - 1) {
```

```
      char* newArr = new char[capacity * 2];
      for (int i = 0; i <= topIndex; i++) newArr[i] = arr[i];
      delete[] arr;
      arr = newArr;
      capacity *= 2;
    }
    arr[++topIndex] = c;
  }

  char pop() {
    if (isEmpty()) throw runtime_error("Stack underflow!");
    return arr[topIndex--];
  }

  bool isEmpty() const { return topIndex == -1; }
};

bool isBalanced(const string& expr) {
  Stack s;
  for (char c : expr) {
    if (c == '(' || c == '[' || c == '{') {
      s.push(c);
    }
    else if (c == ')') {
      if (s.isEmpty() || s.pop() != '(') return false;
    }
    else if (c == ']') {
      if (s.isEmpty() || s.pop() != '[') return false;
    }
    else if (c == '}') {
      if (s.isEmpty() || s.pop() != '{') return false;
    }
  }
  return s.isEmpty();
}

int main() {
  cout << "Q.5: Balanced Symbols Check" << endl;

  string expr1 = "[{(a+b)/(a-b)}+{{d*(e+f)}]";
  string expr2 = "[{(a+b)/(a-b)+(d*f)}+{{d*(e+f)}}]";

  cout << "Expression (a): " << expr1 << " -> "
    << (isBalanced(expr1) ? "Balanced" : "Not Balanced") << endl;

  cout << "Expression (b): " << expr2 << " -> "
    << (isBalanced(expr2) ? "Balanced" : "Not Balanced") << endl;
}
```

Output:

```
Q.5: Balanced Symbols Check
```

Expression (a): [{(a+b)/(a-b)}+{{d*(e+f)}] -> Not Balanced

Expression (b): [{(a+b)/(a-b)+(d*f)}+{{d*(e+f)}}] -> Balanced